UNIVERSIDAD EAFIT · Ingeniería de Sistemas **SI3003 — Inteligencia Artificial**

Guía de apoyo — Parte II (aplicada)

Qué reutilizar de los notebooks de clase, qué adaptar y qué debe preparar

Prof. Jorge Iván Padilla Buriticá

Idea central. En la Parte II puede usar IA y **reutilizar el código de los notebooks de clase**. Los notebooks le dan la infraestructura (clientes, embeddings, lectura de resultados); el examen califica que usted **adapte, verifique e interprete**. Copiar un laboratorio sin adaptarlo no basta.

1. Los cuatro notebooks de clase

Notebook	Qué contiene y le sirve en el examen
`Lecture_08_Yolo_training`	Descarga de datos con Roboflow, entrenamiento con `model.train(...)`, lectura de resultados (`r.boxes`, `box.cls`, `box.conf`, `box.xyxy`).
`01_rag_nvidia_nim`	`EmbeddingService` (con `input_type`), `GenerationService`, `cosine_sim`, `FAISSVectorStore`, `simple_chunker` y `semantic_chunker`, Qdrant, `ask`, `similarity threshold`, evaluación con DeepEval.
`02_local_remote_hf_chatbot_lab`	Modelo local con `transformers` (carga e inferencia), `InferenceClient` remoto de Hugging Face, chat con memoria.
`03_nvidia_chatbot_lab`	Cliente `OpenAI` apuntando a NVIDIA, estructura de la respuesta (`usage`), chat con y sin memoria, ventana de contexto, `trim_conversation`.

2. Mapa: sección del examen → notebook → qué hacer

Sección	Notebook y parte	Reutilice	Adapte / cuidado
A.1	Ninguno	`torch.nn.Conv2d` para validar su fórmula	La fórmula y la red de la Pregunta 1 las escribe usted.
A.2	`Lecture_08` (salida del entrenamiento)	Saber qué es un `results.csv` y qué columnas trae	Los nombres traen espacios: `df.columns = df.columns.str.strip()`. mAP50 es `metrics/mAP50(B)`.
A.3	`Lecture_08` (celda de `r.boxes`)	Extraer clase, confianza y caja `xyxy` de cada detección	Los JSON del examen ya tienen ese formato. `iou`, `nms` y el emparejamiento los programa usted.
B.1	Ninguno	—	Prepárese con el taller (B2.1). Pruebe con matrices pequeñas.
B.2	`02` §1.1–1.2 (cargar modelo local)	`AutoTokenizer`, `AutoModelForCausalLM`, `.eval()`	Use `gpt2` con prompt crudo (sin `chat_template`) y tome `logits[0,-1]`. Groq no devuelve `logprobs`.
B.3	`03` §3 y §9 (`usage`, ventana de contexto)	Idea de contar tokens	`tiktoken` no está en los notebooks.
C.0.1	`03` §1–2 y `02` §2	`OpenAI(base_url,api_key)` y `chat.completions.create`	Cambie `base_url` por proveedor. `enable_thinking` es solo de NVIDIA; Groq rechaza `logprobs`.
C.0.2–3	`01` §0.2 y §2.1	`EmbeddingService`, `cosine_sim`, la idea de `input_type`	Los embeddings del examen vienen del **modelo oficial** (ver `embedding_info.json`): no los mezcle con otro modelo.
C.1	`01` §2.2 y §4 (`search`, `retrieve`)	La lógica de top-k	Sin FAISS: con vectores normalizados, coseno = producto punto (`E @ q`).
C.2	Ninguno	`load_documents` (formato del corpus)	BM25 (`rank_bm25`) y RRF los escribe usted.
C.3	`01` §2.2 (`IndexFlatIP`)	`faiss.normalize_L2`, `IndexFlatIP` como verdad exacta	HNSW es nuevo: `IndexHNSWFlat(D,16,...)` y `hnsw.efSearch`.
C.4	`01` §4–6 (`ask`, prompt, DeepEval)	Plantilla de prompt, `GenerationService.generate`	El contexto son los 3 chunks de **su** `retrieve`. Fidelidad a mano; DeepEval es opcional (cuota).

Lo que no está en ningún notebook (hay que prepararlo desde teoría y taller): `conv_out`, `iou`, `nms`, atención causal, temperatura/entropía, tokenización con `tiktoken`, BM25/RRF, HNSW y la verificación de compatibilidad de embeddings.

3. Qué debe tener listo *antes* del examen

☐ **Una clave de chat funcionando** en *Colab Secrets*: `NVIDIA_API_KEY`, `GROQ_API_KEY` o `HF_TOKEN`. Hágale un *ping* y anote el **nombre exacto del modelo** que funcionó (use `client.models.list()`).

☐ **Modelos locales descargados**: `gpt2`, el modelo de embeddings oficial y `all-MiniLM-L6-v2` (para el contraejemplo de C.0.3), más el codificador de `tiktoken`. El notebook base trae una celda de "precalentamiento".

☐ Acceso a la carpeta de Drive y a `datos_examen/` dentro de Colab.

☐ Sus notebooks de clase **a la mano** (abiertos en otra pestaña), sin claves escritas en ellos.

☐ Repasados el taller de preparación y las reglas de compatibilidad R1–R4 del enunciado.

4. Qué debe generar y guardar en un "cuaderno de utilidades" propio

Funciones cortas, probadas y **sin claves**, que pueda pegar en el examen y adaptar en minutos:

1. `make_chat_client(provider)`: un `OpenAI(base_url=..., api_key=...)` para NVIDIA, Groq y Hugging Face, con las URL del enunciado.
2. `chat(client, model, prompt, provider)`: aísla los parámetros específicos de proveedor (p. ej. `extra_body` solo para NVIDIA).
3. `cosine_sim(a, b)` y una versión matricial `E @ q`.
4. Una función `recall_at_k(...)` y `mrr(...)` reutilizable para C.1 y C.2.
5. Un lector de `results.csv` que haga `strip()` de columnas y grafique pérdida y mAP50.
6. Una plantilla de tabla de afirmaciones para calcular Faithfulness a mano.

5. Cómo adaptar el código de clase al examen (compatibilidad)

- **El chat es intercambiable**: NVIDIA, Groq y Hugging Face usan el SDK `openai`; cambian `base_url`, clave y modelo. **Groq y el router de Hugging Face no ofrecen embeddings.**
- **El LLM es independiente del embedding**: puede generar con Groq y embeber con un modelo local.
- **Mismo modelo para indexar y consultar** (R1). Misma dimensión *no* significa compatible (R2).
- Si usa NVIDIA para embeddings, `input_type` es `passage` para documentos y `query` para consultas; en otros proveedores ese parámetro puede dar error.
- Los embeddings que se entregan están **normalizados**: no vuelva a normalizarlos distinto ni los compare con vectores de otro modelo.

6. Plan sugerido para los 60 minutos

Sección	Minutos	Cómo aprovechar los notebooks
Configuración	3	Una sola celda de datos personales y claves del notebook base.
A	18	Cargue los CSV y los JSON primero; deje `iou`/`nms` para después de las gráficas.
B	15	Atención y tokens son rápidas si ya las practicó; `gpt2` solo necesita los logits.
C	20	Empiece por C.1 (usa lo ya embebido); deje C.4 para el final (necesita red).
D y revisión	4	Declaración de IA, ejecutar todo de nuevo, buscar claves, enviar el correo.

7. Errores frecuentes

- Dejar una API key visible en el código o en una salida: se penaliza con 5 puntos.
- Usar el modelo de embeddings de NVIDIA para consultar contra los embeddings oficiales del examen.
- Pasar `logprobs` a Groq o `extra_body` de NVIDIA a otro proveedor (error 400).
- Olvidar el `strip()` de los nombres de columna del CSV.
- Aplicar NMS sin separar por clase, o emparejar sin exigir la misma clase.
- Calcular softmax con temperatura dividiendo *después* y no sobre los logits.
- Entregar un notebook que no corre de arriba hacia abajo.

Entrega: un notebook ejecutado, por correo a `jipadillab@eafit.edu.co`, **hasta las 11:00 a. m.**
Asunto: `[SI3003-Nombre_Apellido_Grupo_Eval02_practica]`. No incluya claves.